MLOps - Aula 1

Fundamentos, ciclo de vida e maturidade

Plano de voo

- **Quem somos** — apresentação da turma
- **Por que MLOps** — da ciência de dados à engenharia de ML
- **Ciclo de vida** — end-to-end e o “Ops”
- **Versionar e garantir qualidade** — dados, código, modelo, pipeline
- **Maturidade** — níveis, componentes, diagnóstico
- **Atividade em grupo** — diagnóstico de maturidade



01

Por que MLOps

Um modelo no notebook ainda não é um produto

A maior parte dos modelos

não chega à produção

- ou chega e se degrada em silêncio.

O gargalo raramente é o algoritmo.

É tudo o que vem depois: integrar, versionar, implantar, monitorar e manter.



Da ciência de dados à engenharia de ML

Ciência de Dados

estatística, EDA, modelos preditivos

transformar em software

Engenharia de ML

sistemas integrados, escaláveis e em operação

Pergunta: o que os dados dizem?

Pergunta: isso funciona todos os dias?



Seis razões para adotar MLOps

01 **Velocidade**

do desenvolvimento à produção em menos tempo

02 **Qualidade**

CI/CD garante modelos robustos antes do deploy

03 **Escala**

recursos e custos sob controle quando o uso cresce

04 **Monitoramento**

modelos se degradam quando os dados mudam

05 **Colaboração**

cientistas, engenheiros e operações no mesmo fluxo

06 **Governança**

privacidade e conformidade em saúde, finanças, setor público



01 Velocidade

MLOps permite uma transição mais rápida e eficiente de modelos do desenvolvimento para a produção.

Sem esse processo, cada deploy vira um projeto artesanal: o notebook passa de mão em mão, o código é reescrito e o ambiente muda no caminho.

Onde a velocidade de implantação é diferencial competitivo, isso é essencial.

Na prática

- Pipeline automatizado do treino ao deploy
- Ambientes padronizados com contêineres
- Registro de modelos para promover versões (staging → produção)

Como medir

tempo entre o fim do experimento e o modelo em produção



02 Qualidade

Ao incorporar integração contínua (CI) e entrega contínua (CD), MLOps garante que os modelos sejam robustos, seguros e performáticos antes de chegar à produção.

Em ML, testar só o código não basta: dados e modelo também precisam de testes. Por isso surge um terceiro “C”, o treinamento contínuo (CT).

Na prática

- Testes de dados: esquema, faltantes, distribuição
- Testes de modelo: métrica mínima contra um baseline
- Promoção bloqueada se a validação falhar

Como medir

taxa de deploys revertidos (rollback)



03 Escala

MLOps facilita o gerenciamento de recursos computacionais e a escalabilidade das soluções de ML, controlando custo e complexidade à medida que o uso cresce.

Sculley et al. (2015) mostraram que o código do modelo é uma fração pequena de um sistema de ML real. O resto é infraestrutura, dados e configuração, e é aí que a dívida técnica se acumula.

Na prática

- Orquestração e escalonamento horizontal (Kubernetes)
- Serviços gerenciados: SageMaker, Azure ML, Vertex AI
- Reaproveitar pipelines e features entre projetos

Como medir

custo por predição e tempo de resposta sob carga



04 Monitoramento

Em produção, os modelos precisam ser monitorados continuamente para continuar eficazes à medida que os padrões dos dados mudam.

Data drift: muda a distribuição das entradas. Concept drift: muda a relação entre entradas e resultado. Nos dois casos o modelo degrada sem gerar erro, em silêncio.

Na prática

- Métricas de desempenho e de dados em painel
- Alertas por limiar (Prometheus, Grafana, Evidently)
- Retreino disparado por degradação

Como medir

tempo até detectar uma queda de desempenho



05 Colaboração

MLOps aproxima cientistas de dados, engenheiros de ML e profissionais de operações, quebrando silos e melhorando a comunicação.

Isso alinha o desenvolvimento de ML aos objetivos do negócio e às necessidades operacionais. Artefatos compartilhados (registro de modelos, feature store) viram a linguagem comum entre as equipes.

Na prática

- Repositório único com revisão de código
- Registro de modelos visível para todos
- Critério comum de “pronto para produção”

Como medir

número de entregas manuais entre equipes



06 Governança

MLOps cobre governança, privacidade e conformidade regulatória dos modelos, aspectos críticos em saúde, finanças e serviço público.

Governar é conseguir responder, a qualquer momento: quem treinou o modelo, com quais dados, que versão está em produção e por que ela decidiu assim. No Brasil, a LGPD já exige esse cuidado com dados pessoais.

Na prática

- Linhagem de dados e trilha de auditoria
- Model cards documentando uso e limites
- Controle de acesso a dados e modelos

Como medir

tempo para responder a uma auditoria



02

Ciclo de vida

Do problema de negócio ao modelo que se atualiza sozinho

Workflow end-to-end de ML

DADOS

1. Definir o problema
2. Coletar dados
3. Limpar e preparar
4. Explorar (EDA)

MODELO

5. Engenharia de features
6. Dividir treino/val/teste
7. Treinar e selecionar
8. Avaliar (validação)
9. Testar com dados novos

PRODUÇÃO

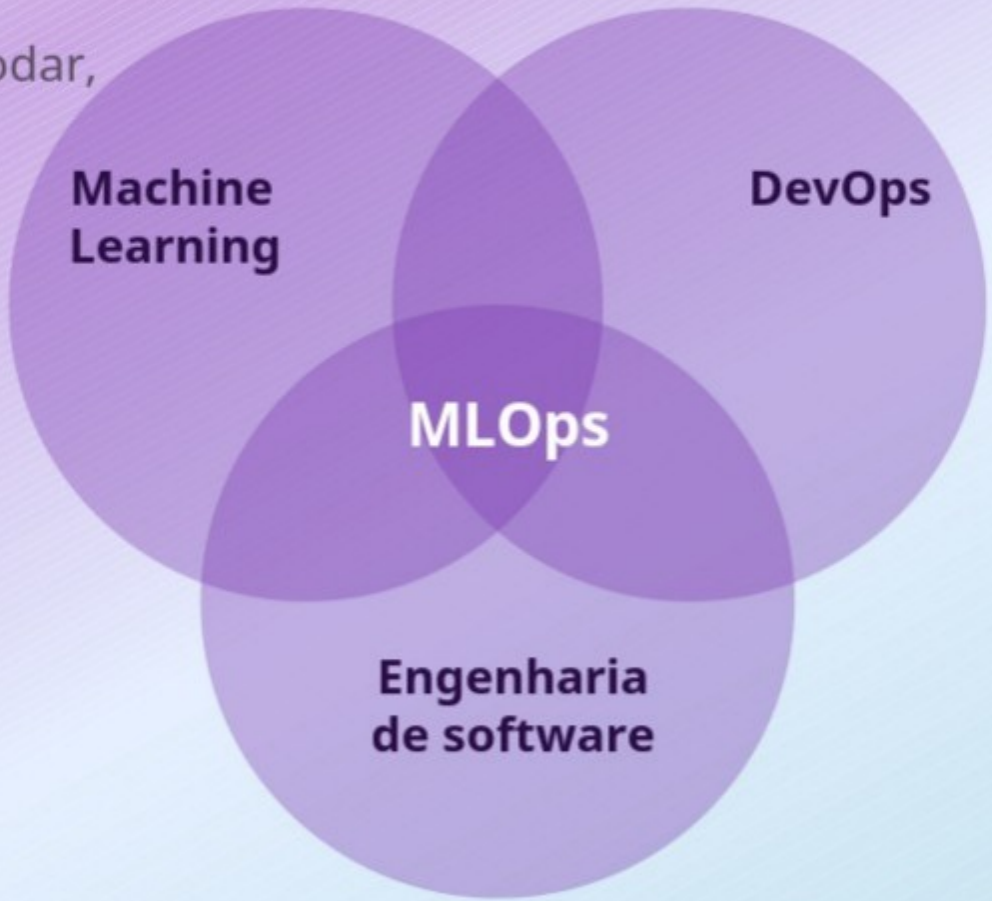
10. Implantar
11. Monitorar e manter
12. Atualizar com novos dados

novos dados realimentam o ciclo



O que significa o “Ops” em MLOps

Ops = operations:
tudo o que leva o modelo a rodar,
e continuar rodando, em
produção.



[redacted]

As seis frentes do “Ops”

CI/CD

testar e liberar modelos de forma automática

Monitoramento

detectar degradação e mudança nos dados

Configuração e infra

ambientes reproduzíveis para treinar e servir

Gestão de dados

pipelines com qualidade e governança

Versão e artefatos

rastrear modelos, dados e código; permitir rollback

Colaboração

uma cultura comum entre ciência, engenharia e operações



03

Versionar e garantir qualidade

Se não dá para reproduzir, não dá para confiar

O que versionar e com o quê

Pipeline

Kubeflow Pipelines
cada etapa rastreável

Modelo

MLflow · Model Registry
parâmetros, métricas e artefato

Código e configuração

Git · YAML/JSON
scripts e hiperparâmetros

Dados

DVC · Delta Lake
datasets e versões no tempo

Docker empacota o ambiente; CI/CD (GitHub Actions, GitLab CI, Jenkins) automatiza tudo isso.



Rastrear um experimento com MLflow

```
import mlflow

with mlflow.start_run():
    mlflow.log_param("max_depth", 6)
    mlflow.log_metric("f1", 0.81)
    mlflow.log_artifact("model.pkl")
```

Cada execução registra
o que entrou (parâmetros),
o que saiu (métricas)
e o que foi gerado (artefatos).

Resultado:
dá para comparar, repetir e auditar.



Qualidade é do sistema, não do modelo

ANTES DO DEPLOY

DEPOIS DO DEPLOY



1	2	3	4	5	6
Métricas	**Dados de entrada**	**Validação**	**Monitoramento**	**Auditoria**	**Segurança**
desempenho, viés, robustez	faltantes, outliers, distribuição	cross-validation, cenários diversos	drift e alertas por limiar	logging, tracking, reprodutibilidade	privacidade e ataques ao modelo

Ferramentas comuns: DVC e Delta Lake (dados) - Kubeflow e MLflow (pipelines) - Prometheus e Grafana (monitoramento) - Docker e Kubernetes (deploy)

[redacted]

Treino e produção: as mesmas features



Mesma definição de feature nos dois caminhos evita o training-serving skew.

[redacted]

04

Maturidade

Onde a sua organização está e qual o próximo passo

Cinco níveis de maturidade em MLOps



Quatro componentes, com três níveis

	Básico	Intermediário	Avançado
Experimentação	ad-hoc, local, sem rastreio	Git + MLflow/DVC	pipelines automáticos (Kubeflow, Vertex AI)
Implantação	manual, scripts	CI/CD parcial (Jenkins, GitLab)	contínua, rollback automático (Kubernetes, Argo CD)
Monitoramento	manual e reativo	alertas básicos (Prometheus, Grafana)	preditivo, com feedback loops
Governança	documentos e planilhas	políticas e auditorias periódicas	conformidade contínua (Collibra, Alation)



MLOps deve ser independente de fornecedor

Linguagem, framework, plataforma e infraestrutura podem mudar.
O processo precisa continuar de pé.

Ganhos: resiliência, interoperabilidade, flexibilidade, escala e custo, sem aprisionamento tecnológico.

Como chegar lá

Contêineres

rodar igual em qualquer lugar

APIs padronizadas

trocar peças sem quebrar o todo

Orquestração

Kubernetes agnóstico à infra

Monitoramento e logging

avaliação constante



Dez boas práticas

01 Automatize o ciclo inteiro

02 Versione dados, código e modelos

03 Monitore e configure alertas

04 Valide a qualidade dos dados

05 Proteja modelos e acessos

06 Garanta reprodutibilidade (Docker)

07 Projete para escalar

08 Revise código em equipe

09 Teste como software

10 Registre e compare experimentos



Recap

MLOps é engenharia e operação, não modelagem.

Sem versão de dados, nada é reproduzível.

Maturidade se mede por componente e evolui um passo de cada vez.

